🌿 Moodful — Complete GitHub Guide

📁 Project File Structure

```
moodful/
├── .github/
|   └── workflows/
|       └── deploy.yml          # Auto-deploy to GitHub Pages
├── public/
|   └── favicon.ico
├── src/
|   ├── constants/
|   |   └── moods.js            # Mood definitions, MOOD_MAP, MONTHS
|   ├── hooks/
|   |   └── useEntries.js       # Custom hook: load/save/delete entries
|   ├── utils/
|   |   ├── analytics.js        # avgMood, moodLabel, calcStreak, distribution
|   |   ├── dateHelpers.js      # todayKey, fmtDate, getLast7Keys, getLast30Keys
|   |   └── storage.js          # loadEntries, saveEntries, upsertEntry,
deleteEntry
|   ├── components/
|   |   ├── MoodPill.jsx        # Reusable mood badge/button
|   |   ├── MoodBarChart.jsx    # SVG bar chart (7-day / 30-day)
|   |   ├── StatCard.jsx        # Small stat box (avg, logged count)
|   |   ├── LogTab.jsx          # Daily mood logging screen
|   |   ├── InsightsTab.jsx     # Charts + distribution analytics
|   |   └── HistoryTab.jsx      # Searchable journal history
|   ├── App.jsx                 # Root shell with tab routing
|   └── main.jsx                # ReactDOM entry point
├── index.html                  # Vite HTML template
├── vite.config.js              # Vite config (base for GitHub Pages)
├── package.json
└── .gitignore
```

🚀 Part 1 — First-Time Setup

Step 1: Install dependencies & run locally

```bash
```

```bash
cd moodful
npm install
npm run dev
# → Open http://localhost:5173
```

Step 2: Initialise Git

```bash
git init
git add .
git commit -m "feat: initial moodful app"
```

Step 3: Create GitHub repo & push

```bash
# Create a NEW repo on github.com first (no README, no .gitignore)
# Then connect and push:
git remote add origin https://github.com/YOUR_USERNAME/moodful.git
git branch -M main
git push -u origin main
```

🌿 Part 2 — Full Day-to-Day Workflow

A) Start a new feature on its own branch

```bash
git checkout -b feat/mood-streak
# Make your code changes…
```

B) Stage & commit your changes

```bash
```

```bash
# See what changed
git status

# Stage specific files
git add src/components/LogTab.jsx
git add src/utils/analytics.js

# Or stage everything
git add .

# Commit with a clear message
git commit -m "feat: add streak counter to log tab"
```

C) Push your branch to GitHub

```bash
git push origin feat/mood-streak
```

D) Open a Pull Request (PR)

1. Go to **github.com/YOUR_USERNAME/moodful**
2. Click **"Compare & pull request"** (banner appears after push)
3. Fill in title + description
4. Click **"Create pull request"**

E) Review, approve & merge the PR

```bash
# On GitHub UI: click "Merge pull request" → "Confirm merge"

# Then locally, update your main branch:
git checkout main
git pull origin main
```

F) Delete the feature branch (cleanup)

```bash
```

```
# Delete locally
git branch -d feat/mood-streak

# Delete on GitHub
git push origin --delete feat/mood-streak
```

🔁 Part 3 — Common Daily Commands

Task	Command
Check status	`git status`
See commit history	`git log --oneline`
See what changed	`git diff`
Undo unstaged changes	`git restore src/App.jsx`
Undo last commit (keep files)	`git reset --soft HEAD~1`
Stash work-in-progress	`git stash`
Restore stash	`git stash pop`
Pull latest from main	`git pull origin main`
List all branches	`git branch -a`
Switch branch	`git checkout feat/history-search`

🌐 Part 4 — Deploy to GitHub Pages

Option A: Automatic (CI/CD via GitHub Actions) ✅ Recommended

The `.github/workflows/deploy.yml` file already handles this. Every push to `main` triggers a build + deploy automatically.

One-time setup:

1. Go to your repo → **Settings** → **Pages**

2. Under **Source**, select **"GitHub Actions"**
3. Push any commit to `main` — it deploys itself!
4. Your app is live at: `https://YOUR_USERNAME.github.io/moodful/`

Option B: Manual deploy (gh-pages package)

```bash
# Install the deploy helper
npm install --save-dev gh-pages

# Add to package.json scripts:
# "predeploy": "npm run build",
# "deploy":    "gh-pages -d dist"

# Add homepage to package.json:
# "homepage": "https://YOUR_USERNAME.github.io/moodful"

# Deploy manually anytime:
npm run deploy
```

🌳 Part 5 — Branch Strategy (Recommended)

```
main                    ← production, always deployable
├── feat/journal-export ← new feature
├── feat/dark-mode      ← another feature
├── fix/streak-bug      ← bug fix
└── chore/update-deps   ← maintenance
```

Naming conventions

Type	Prefix	Example
New feature	`feat/`	`feat/mood-export`
Bug fix	`fix/`	`fix/chart-overflow`
Refactor	`refactor/`	`refactor/storage-hook`

Type	Prefix	Example
Style/UI tweak	`style/`	`style/pill-animation`
Maintenance	`chore/`	`chore/update-vite`

💡 Part 6 — Good Commit Message Conventions

```bash
# Format: type(scope): short description

git commit -m "feat(log): add emoji mood picker"
git commit -m "fix(chart): prevent overflow on small screens"
git commit -m "refactor(storage): extract upsertEntry helper"
git commit -m "style(pill): soften selected state shadow"
git commit -m "chore: update vite to v5.1"
git commit -m "docs: add GitHub setup guide"
```

⚡ Part 7 — Quick Reference Cheatsheet

```bash
```

```
# ── SETUP ──────────────────────────────────────────
git init                              # Start a new repo
git clone <url>                       # Clone existing repo
git remote add origin <url>           # Connect to GitHub

# ── DAILY WORK ─────────────────────────────────────
git status                            # What's changed?
git add .                             # Stage all changes
git commit -m "feat: add feature"     # Commit
git push origin main                  # Push to GitHub
git pull origin main                  # Pull latest

# ── BRANCHES ───────────────────────────────────────
git checkout -b feat/my-feature       # Create + switch branch
git checkout main                     # Switch to main
git merge feat/my-feature             # Merge branch into current
git branch -d feat/my-feature         # Delete branch locally
git push origin --delete feat/my-feature  # Delete on GitHub

# ── UNDO ───────────────────────────────────────────
git restore <file>                    # Discard file changes
git reset --soft HEAD~1               # Undo last commit (keep changes)
git stash                             # Save WIP temporarily
git stash pop                         # Restore WIP

# ── HISTORY ────────────────────────────────────────
git log --oneline                     # Compact commit log
git diff                              # Unstaged changes
git diff --staged                     # Staged changes
```

Happy shipping! 🌿